```python
# routes.py
from flask import Flask, render_template, request, jsonify
import sqlite3
import os
import traceback
from Bracket.simulate_game_engine import (simulate_game, Team, load_team_data_from_db, init_score_by_round,
get_models_and_data)
from Bracket.simulate_bracket_engine import simulate_tournament

# Import data needed for projects
conn = sqlite3.connect('bracket_sim.db')
conn.close()

def register_routes(app):
    @app.route('/')
    @app.route('/home')
    @app.route('/bio')
    def home_page():
        return render_template('bio.html')

    @app.route('/resume')
    def resume():
        return render_template('resume.html')

    @app.route('/projects')
    def projects():
        return render_template('projects.html')

    @app.route('/Game_Sim')
    def game_sim():
        return render_template('Game_Simulator.html')

    @app.route('/simulate', methods=['POST'])
    def simulate():
        from Bracket.simulate_game_engine import init_score_by_round, simulate_game
```

```

        try:
            print("\n--- /simulate called ---")
            db_path = os.path.join(app.root_path, "data", "bracket.db")

            team1_name = request.form.get('team1')
            team2_name = request.form.get('team2')
            if not team1_name or not team2_name:
                return jsonify({"error": "Both teams must be selected"}), 400

            noise_std = float(request.form.get('noise_std', 0))
            consistency_multiplier = float(request.form.get('consistency_multiplier', 1.0))
            normalize_scores = request.form.get('apply_normalization', 'False') == 'True'

            df, model, EXPECTED_FEATURES = get_models_and_data(
                db_path, [team1_name, team2_name]
            )

            seed1 = int(df[df["Team"] == team1_name]["Seed"].values[0])
            seed2 = int(df[df["Team"] == team2_name]["Seed"].values[0])
            team1 = Team(name=team1_name, seed=seed1)
            team2 = Team(name=team2_name, seed=seed2)

            # 🔍 Insert your debug prints here, right before simulate_game()

            init_score_by_round()
            result = simulate_game(
                team1, seed1,
                team2, seed2,
                db_path=db_path,
                model=model,
                expected_features=EXPECTED_FEATURES,
```

```
                noise_std=noise_std,
                consistency_multiplier=consistency_multiplier,
                blend_weight=float(request.form.get('blend_weight', 0)),
                normalize_by_round=normalize_scores
            )

            xgb_line = f" {team1.name} ({result['t1_score']:.1f}) vs {team2.name} ({result['t2_score']:.1f}) → Winner: {result['winner'].name}"
            return jsonify({"XGB_result": xgb_line})

        except Exception as e:
            traceback.print_exc()
            return jsonify({"error": str(e)}), 500

        finally:
            print("--- /simulate finished ---\n")

    @app.route('/XGB_Sim')
    def XGB_sim():
        return render_template('XGBoost_Bracket_Sim.html')

    @app.route("/simulate_bracket", methods=["POST"])
    def simulate_bracket():
        try:
            print("\n--- /simulate_bracket called ---")
            print("Form data:", dict(request.form))

            noise_std = float(request.form.get("noise_std", 1.0))
            consistency_multiplier = float(request.form.get("consistency_multiplier", 1.0))
            blend_weight = float(request.form.get("blend_weight", 0.0))
            normalize_by_round = request.form.get("apply_normalization", "").lower() == "true"
```

```python

            print(
                f"Parameters: noise_std={noise_std}, consistency={consistency_multiplier}, blend={blend_weight}, normalize={normalize_by_round}")

            num_sims = int(request.form.get("num_sims", 1))

            all_logs = []
            champions = []

            for sim_num in range(num_sims):
                print(f"Running simulation {sim_num + 1}/{num_sims}")
                result = simulate_tournament(
                    noise_std=noise_std,
                    consistency_multiplier=consistency_multiplier,
                    blend_weight=blend_weight,
                    normalize_by_round=normalize_by_round
                )
                print(
                    f"Simulation {sim_num + 1} result: Champion = {result['champion_name']} (#{result['champion_seed']})")

                all_logs.append(result["log"])
                champions.append((result["champion_name"], result["champion_seed"]))

            response_data = {
                "champions": champions,
                "logs": all_logs
            }

            print(f"Returning response with {len(champions)} champions and {len(all_logs)} logs")
            return jsonify(response_data)

```

```
        except Exception as e:
            print(f"Error in simulate_bracket: {str(e)}")
            traceback.print_exc()
            return jsonify({"error": str(e)}), 500

    @app.route('/RanSel_Sim')
    def ransel_sim():
        return render_template('Random_Selection_Bracket_Sim.html')
```